Adaptive Polygon Simplification Basing on Delaunay Triangulation and its Application in High Speed PCBs and IC Packages Simulation

Zhanghong Tang[1], Jun Zhu[2], Fei He[2], Linwei Feng[2], Guohua Yang[2], Guanghui Han[3]

[1]College of Material Science and Engineering, Beijing University of Technology, Beijing 100124, China
[2]China Petroleum Logging (CPL) Co., Ltd., Xi'an, Shannxi 710054, China
[3]China Petroleum & Chemical Co., Ltd., Shaanxi Petroleum Branch, Xi'an, Shannxi 710003, China

As modern high speed Package Circuit Boards (PCBs) and Integrated Circuit (IC) packages become more and more complicated, the electromagnetic simulation inside the PCBs and IC packages becomes more and more difficult. The most challenge work is that complicated multiple layer shapes inside PCBs and IC packages need very huge amount of high quality meshes for electromagnetic simulation, which lead to huge memory requirement for solving the linear matrix. For most personal computers, that requirement usually lead to out of memory in matrix solution. To overcome this problem without losing the simulation precision, this article presents an algorithm to simplify complex 2-D polygons based on Delaunay triangulation. By checking the smallest angles of triangles which contain an edge of the polygons, the algorithm simplifies polygons adaptively and updates the simplified polygons' Delaunay triangulation. Different from other simplifications which try to decrease the count of vertices, this algorithm tries to delete redundant short edges, leading to greatly reduced mesh points during the process of generating the constrained and quality Delaunay triangulation. The algorithm presented here has been applied for mesh generation for finite-element analysis of complicated PCBs and IC packages. Test results show that the proposed algorithm can reduce more than 75% in total nodes in comparison with the approach without simplification, resulting in significant reduction of computer resources in numerical computation.

***Index Terms*—Adaptive Polygon Simplification, Delaunay Triangulation, High Speed PCBs and IC Packages, Finite Element Method**

I. Introduction

The finite element method (FEM) has been widely adopted in the electromagnetic simulation of high speed Printed Circuit Boards (PCBs) and Integrated Circuit (IC) packages, where the 2-D structure of every layer (often described by polygons) is discretized through constrained Delaunay triangulation for FEM modeling. However, as modern high speed PCBs and IC packages have more and more layout layers and the shapes of every layer are more and more complicated, the electromagnetic simulation inside the PCBs and IC packages becomes more and more difficult since very huge amount of high quality meshes are needed, which lead to huge memory requirement for solving FEM linear matrix. On the other hand, due to the extreme complexity of shape structures, the original layout data of every layer may contain a huge number of redundant short edges that will lead to excessive number of FEM nodes when generating high quality triangle meshes. A proper polygon simplification process that removes those redundant edges is therefore highly desired to reduce the consequent FEM computational resources. The purpose of polygon simplification is to remove as many redundant edges of polygons as possible while the changes of shapes and the field distribution are very small.

Previous works on the polygon simplification are either not linear time in the computational complexity[1], or only aimed at minimizing the number of inflections, for example the work in [2]. Most of those algorithms are not suitable for generating FE mesh, since they did not try to delete the short edges of a polygon that may lead to excessive count of high quality mesh elements. Besides, those algorithms only employs a constant minimal distance threshold during the polygon simplification for all polygons, as a result, it may cause polygon overlapping when the separation between the two polygon's edges is less than the pre-defined threshold. The polygon overlapping will lead to the change of circuit path, a very serious problem in electromagnetic simulation of PCBs and IC packages.

To address the above problems, this paper introduces a new polygon simplification method that is based on Delaunay triangulation of polygon vertices. Unlike the previously proposed algorithms, the approach presented here focuses on removing edge-associated extremely low quality triangles (where the minimal-angle is smaller than a pre-defined threshold) from the initial Delaunay triangulation. This new method simplifies the polygon dynamically as the Delaunay triangulation updates, is able to remove extremely short edges and avoid the polygon overlapping. Principles and numerical examples will be presented in the following section, and the results show that the FEM node counts can be reduced up to 75% by employing the new polygon simplification approach while nearly identical simulation results are obtained.

II. Current Distribution Simulation of PCBs and IC Packages

One of most important factors to decide the layout design of PCBs and IC packages is the current distribution of every layer. Designers often hope the current distribution in layers to be a reasonable range. Another key feature that designers are interested is the IR drop between the voltage source and ports in PCBs and IC packages, which can be used to design the power supply system of PCBs and IC packages to ensure that all components have enough voltage margins to work correctly.

When a steady current flows in metal layout (the metal layers of the PCBs and IC packages), the electrical potential in the non-source region satisfies Laplace's equation for the scalar electrical potential[3]:

$$\frac{\partial}{\partial x}\left(\sigma(x,y,z)\frac{\partial\varphi}{\partial x}\right)+\frac{\partial}{\partial y}\left(\sigma(x,y,z)\frac{\partial\varphi}{\partial y}\right)+\frac{\partial}{\partial z}\left(\sigma(x,y,z)\frac{\partial\varphi}{\partial z}\right)=0 \tag{1}$$

with the boundary conditions

$$\begin{cases}\varphi|_{\Gamma}=u\\ \sigma(x,y,z)\dfrac{\partial\varphi}{\partial n}=J_n\end{cases} \tag{2}$$

where (x,y,z) is the coordinate of a point in the domain, $\sigma(x,y,z)$ is the conductivity of point (x,y,z), J_n is the volume current density due to the external source, and φ is the electrical potential in the non-source domain to be solved.

To solve the problem given by Eqs. (1) and (2), a general FEM is applied on layout of every metal layer at first to form the equations of the steady-state current field[4], which needs to discretize the complicated metal shapes of metal layers to small elements. Usually small and high quality elements could result high precision, however, locally too small elements will lead to large memory requirement without improving the simulation precision. A better solution is to remove redundant short edges of polygons given in the layout data and then generate high quality elements basing on the simplified polygons.

III. Polygon Simplification Approach

The purpose of polygon simplification is to delete as many redundant edges of polygons as possible without losing the precision of shapes. In Delaunay triangulation, an edge of polygon is called a lost edge if there is no triangle associated with the polygon edge. On the contrary, the edge will be called an associated edge if it is an edge of some triangle of current Delaunay triangulation. The simplification approach in this paper supposes that only the associated edges in Delaunay triangulation could be deleted for simplification.

In this paper, a new polygon simplification approach was presented based on the Delaunay triangulation results. Unlike the previously proposed algorithms, the approach focuses on removing edge-associated extremely low quality triangles from the initial Delaunay triangulation. This new approach simplifies the polygon dynamically as the Delaunay triangulation updates, and is able to remove extremely short edges and avoid the polygon overlapping.

A. Select might-be-deleted Edges

Before simplification, the edges that might be deleted are firstly selected from all edges and pushed into a queue basing on the Delaunay triangulation mesh. The rules to select might-be-deleted edges are as follows.

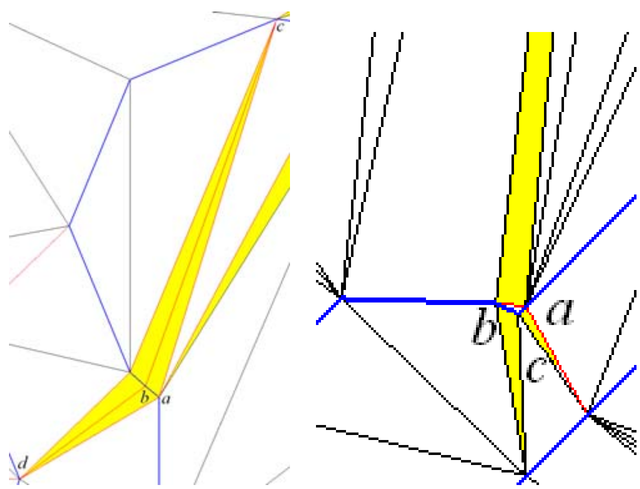
Rule 1: *For an edge*

If the edge is shared by two triangles in bad quality (e.g., edge *ab* in Fig. 1 (a)), and the two angles corresponding to the edge (angle *acb* and *abd*) are smallest angles of their triangles angles, the edge should be selected. The bad quality triangle means its smallest angle is less than a pre-defined threshold.

Rule 2: *For two adjacent edges*

If two edges are the sides of a triangle (e.g., edge *ac*, *cb* and triangle *abc* in Fig. 1 (b)), and the triangle's three neighboring triangles are all in bad quality and whose smallest angles corresponding to the triangle's sides, both edges should be selected.

Here bad quality triangle means the smallest angle of the triangle is less than pre-defined threshold (less than the required minimal angle for quality triangulation). In Fig. 2 the edges associated with shadow triangles meet rule 1 and rule 2 respectively.



(a) (b)

Fig. 1. Rules to select might-be-deleted edges, (a) rule 1; (b) rule 2.

B. Polygon Simplification

The process of polygon simplification is based on the queue of edges that are selected as might-be-deleted. For every edge popped up from the queue, the algorithm firstly searches for the shortest edge from the continuous pushed in edges around the popped up edge and decide whether it should be deleted. The process of the algorithm is as follows.

Polygon simplifying based on Delaunay triangulation:

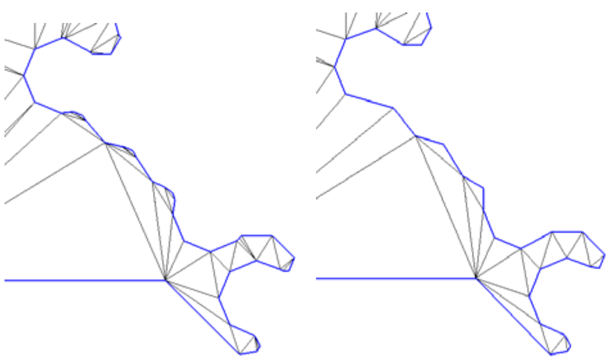
1. *Select might-be-deleted edges:*
2. *Push in the edges which meet rule 1 or rule 2 to a queue* ***Q****.*
3. *Simplify polygon and update triangulation:*
4. *Loop*
    5. *If (****Q*** *is empty) jump out of the loop;*
    6. *Let* ***bnd****=pop(****Q****), search a shortest edge* ***bndshort*** *among the continuous edges around* ***bnd****, push in* ***bnd*** *to* ***Q*** *again and delete* ***bndshort*** *from* ***Q****;*
    7. ***Bndshort*** *is to be deleted at this step. Each of its end vertex and the vertex's two adjacent vertices form a triangle which associates with two adjacent edges. If two adjacent edges (including* ***bndshort****) associate a smaller area triangle, they will be replaced by a new edge* ***bndnew.*** *However, if one of the two adjacent edges is a lost edge, another group of adjacent edges will be replaced;*

8. GOTO End Loop;
9. *Delete the two adjacent edges from* ***Q*** *if they have been pushed into* ***Q****;*
10. *Update Delaunay triangulation after deleted* ***bndshort****;*
11. *If* ***bndnew*** *still should be selected, push it into* ***Q****;*
12. *End Loop*

Fig. 2. Flow of adaptive polygon simplification based on delaunay triangulation

In Fig. 2, when an edge popped out from the queue, the approach firstly searches for the shortest edge in the queue from the continuous edges around it. For example, edge number 5 is popped out, the approach firstly searches for the shortest edge from the continuous edges around edge 5 in the queue. Suppose edges 2, 3, 4, 6, 7 of the polygon also exit in the queue, the shortest edge should be found among these edges (it is supposed that neighboring edges of polygons are numbered continuous). However, if the only edges 2, 3, 7 exist in this queue, no edge could be selected since there isn't any edge being the neighbor of edge 5 (neighbor edge of 5 would have a common vertex with edge 5). If a shortest edge was selected, it should be deleted from the polygon and the queue. The process of deleting the edge from the polygon is simply deleting one of the vertices according to a smaller area change. After deleting a vertex, another edge sharing the vertex with the deleted edge is changed. The changed edge should be pushed into the queue if it is still might-be-deleted.

Fig. 3 is an example of polygon simplification results, in which only a local part of the polygon is given. Fig. 3 (a) is the original polygon and its Delaunay triangulation, and Fig. 3 (b) is the simplification and the triangulation.

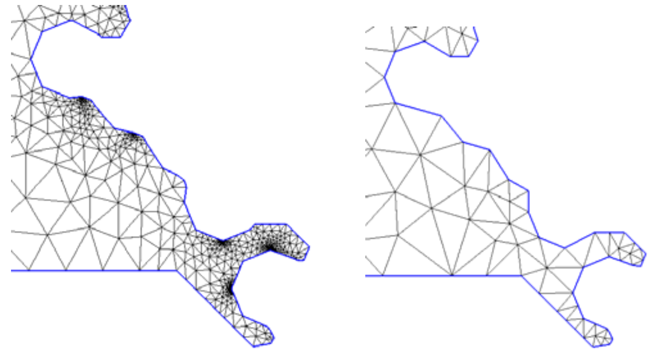


(a) (b)

Fig. 3. An example of polygon simplification basing on Delaunay triangulation, (a) original polygon, (b) simplified polygon.

By comparing Fig. 3(a) and Fig. 3(b) it can be found that the shapes have very small changes which can make sure that the simulation results almost have no change. On the other hand, the simplification shown in Fig. 3(b) has removed many redundant short edges that could results much less elements under the same mesh quality condition.

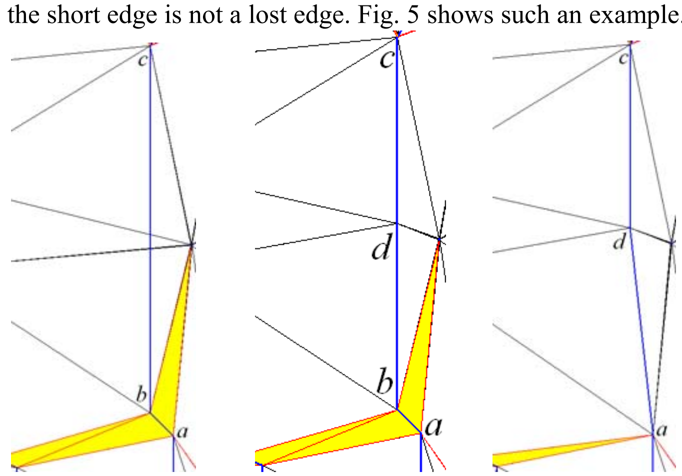
Fig. 4 shows a comparison of the refinement of Delaunay triangulation for the polygon before and after simplification under the same mesh quality condition (the minimal angle of triangle is no less than 30°), in which interior nodes are created to get mesh elements in such high quality. It can be found that the number of elements is reduced greatly after the simplification.



(a) (b)

Fig. 4. Final triangulation, (a) before simplified, (b) after simplified.

In Fig. 2, When a vertex of a short edge is selected to be deleted, another edge which shares the vertex with the short edge can't be a lost edge (a lost edge of a polygon is the edge which doesn't appear in Delaunay triangles' edges). However, if the edge is a lost edge, we keep inserting its midpoint into Delaunay triangulation until the edge sharing the vertex with the short edge is not a lost edge. Fig. 5 shows such an example.



(a) (b) (c)

Fig. 5. An example of recovering lost edge before polygon simplification, (a) original polygon, (b) a midpoint is inserted, (c) simplified result.

In Fig. 5(a), a lost edge *bc* shares the vertex *b* with the short edge *ab*. Before deleting vertex *b*, the midpoint of edge *bc* is inserted into Delaunay triangulation until edge *bd* is not a lost edge, which is shown in Fig. 5(b). Fig. 5(c) shows the last simplified result after deleted vertex *b*.

C. Update Delaunay Triangulation

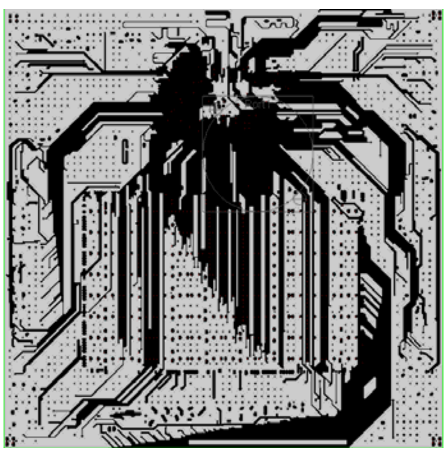
When a vertex is deleted from the meshed polygon, the corresponding Delaunay triangulation should be updated. A usual way is to find star-shape polygon after deleting the vertex and then triangulate the polygon by ear-cutting algorithm[5]. However, we adopted a simple way to fulfill it. Once an edge is certain to be deleted, the triangulation can be updated by moving all concerning triangles' vertices from the deleted vertex to another. In addition, the edge swap process is needed for those triangles with the changed vertices to keep the triangulation's Delaunay property [6].

It is worth to be noticed that the polygon simplification method described in this paper can avoid polygon overlapping automatically. When edges of two polygons are closed each other, the triangles around these edges would have not too bad qualities since the Delaunay triangulation method always find nearest nodes to form triangles, as a result, these edges would never be the might-be-deleted edges.

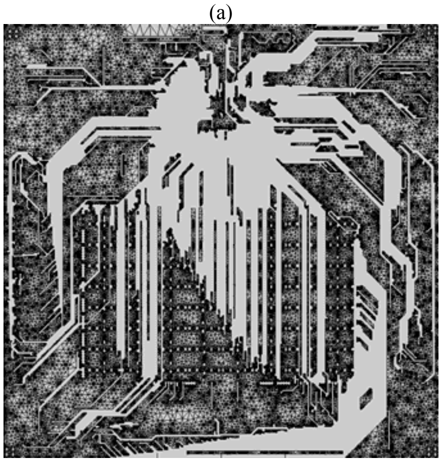
IV. Application

The polygon simplification method is successfully applied to analyze the IR drop of complicated PCBs and IC packages by finite element (FEM) method [3]. Results show that without polygon simplification, some of the PCBs and IC packages models can't be analyzed in 32 bit machines because of out of memory when solution. Even though recently the computers are more and more powerful and 64 bit machines are very common, the FEM matrices solution time will increase greatly if the polygons are not simplified before mesh refinement.

Fig. 6(a) shows a real PCB layer with very complicated metal shapes. Fig. 6(b) shows the refined Delaunay mesh with polygon simplified. In the simulation, a port with a pair of nodes is defined to calculate the resistance between the pair of nodes, which can show the IR drop between the pair of nodes defined by the port.



(a)



(b)

Fig. 6. Polygon simplification and its refined mesh, (a) the shape of a PCB layer, (b) refined mesh based on the simplified polygons.

Table 1 listed the comparison between the simulation results with and without polygon simplification. Results show that by polygon simplification, the size of vertices (unknowns) decreased to less than 1/4 of original size, but the difference between two simulations is less than 0.1%.

TABLE 1 MESH SIZE AND CALCULATION RESULT AFFECTED BY POLYGON SIMPLIFICATION

Refine Method	Nodes	Triangles	Resistance between two ports (Ω)
Without simplification	378,621	700,002	0.1412744
With simplification	81,630	124,194	0.1407982

V. Conclusion

In this article, an adaptive polygon simplification based on Delaunay triangulation was presented, which can automatically recognize polygons' geometric size and shapes as well as getting an efficient simplification without losing the simulation precision. In addition, this method can also avoid polygons overlapping during polygon simplification. Tested by some PCBs and IC packages models, this simplification is especially suitable for mesh generation of large and complicated PCBs and IC packages. It is tested that after applied this new approach, the total FEM nodes decreased for more than 75% for most of the PCBs and IC packages models while their simulation results are nearly unchanged.

Acknowledgment

This work was supported by the Doctor Research Fund of School (No. X0009011200905).

References

[1] U. Ramer, "An iterative procedure for the polygonal approximation of plane curves," *Compute. Gr. and Image Proc.*, 1(3): 244-256, Nov. 1972

[2] John D. Hobby, "Polygonal Approximations that Minimize the Number of Infections," *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms,* 1993, pp. 93-102.

[3] Zhanghong Tanga, and Jiansheng Yuan, "Combination Approach of FEM and Circuit System in IR Drop Analysis and Its Applications," *Tsinghua Science & Technology*, 2008, 13(6):850-857.

[4] Jin Jianming. *The Finite Element Method in Electromagnetics*. New York: John Wiley & Sons, Inc., 1993.

[5] Mir Abolfazl Mostafavi, Christopher Gold and Maciej Dakowicz, "Delete and insert operations in Voronoi/Delaunay methods and applications, " *Computers & Geosciences 29*, 2003, pp. 523–53.

[6] Edelsbrunner, Herbert; Nimish Shah, "*Incremental Topological Flipping Works for Regular Triangulations*", 1996, Algorithmica 15: 223–241.